
ECSE 4660/6660: Internetworking of Things

Lecture 17: Network Layer – Part V:
Dijkstra, 6LowPAN and RPL

Ish Jain
ECSE, RPI

Spring 2025

Administrative

- Use Piazza for any questions
 - <https://piazza.com/rpi/spring2026/ecse46606660>
- Today's Quiz is available on **LMS**
 - Multiple choice questions
 - Due tonight 11:59 PM
- Lab
 - TA hours 4-5 PM daily in 5312 lab
 - Lab 4 due Fri, 3/20 (DSR, AODV, ARP)
- HW
 - HW3 released, due Mon, 3/16
 - No Late submission allowed
- Remote class 2/26, ~~3/23~~ 3/30

- Project group formation
 - [IoT_SP26_project.xlsx](#)
- Project Next Steps
 - Mid-term report due **3/25** (<1 week)
 - In-class presentation+2min video 4/20, 4/23
 - Live-demo 4/27
 - Final code/report 5/1

Network layer: “data plane” roadmap

- Network layer: overview
- IP: the Internet Protocol
 - Datagram format and IP Addressing
 - DHCP ✓
 - ARP ✓
 - Routing Table Lookup
 - IPv6
 - NAT ✓
 - ICMP
 - ICMPv6
- Routing Protocol for Internet
- Routing Protocol for IoT



Network layer: “control plane” roadmap

- routing algorithms
 - link state - Dijkstra
 - distance vector – Bellman Ford
- Routing protocols
 - OFPF
 - BGP
- IoT Routing Protocols
 - 6LOWPAN
 - RPL



Network-layer functions

- **forwarding**: move packets from router's input to appropriate router output

data plane

- **routing**: determine route taken by packets from source to destination

control plane

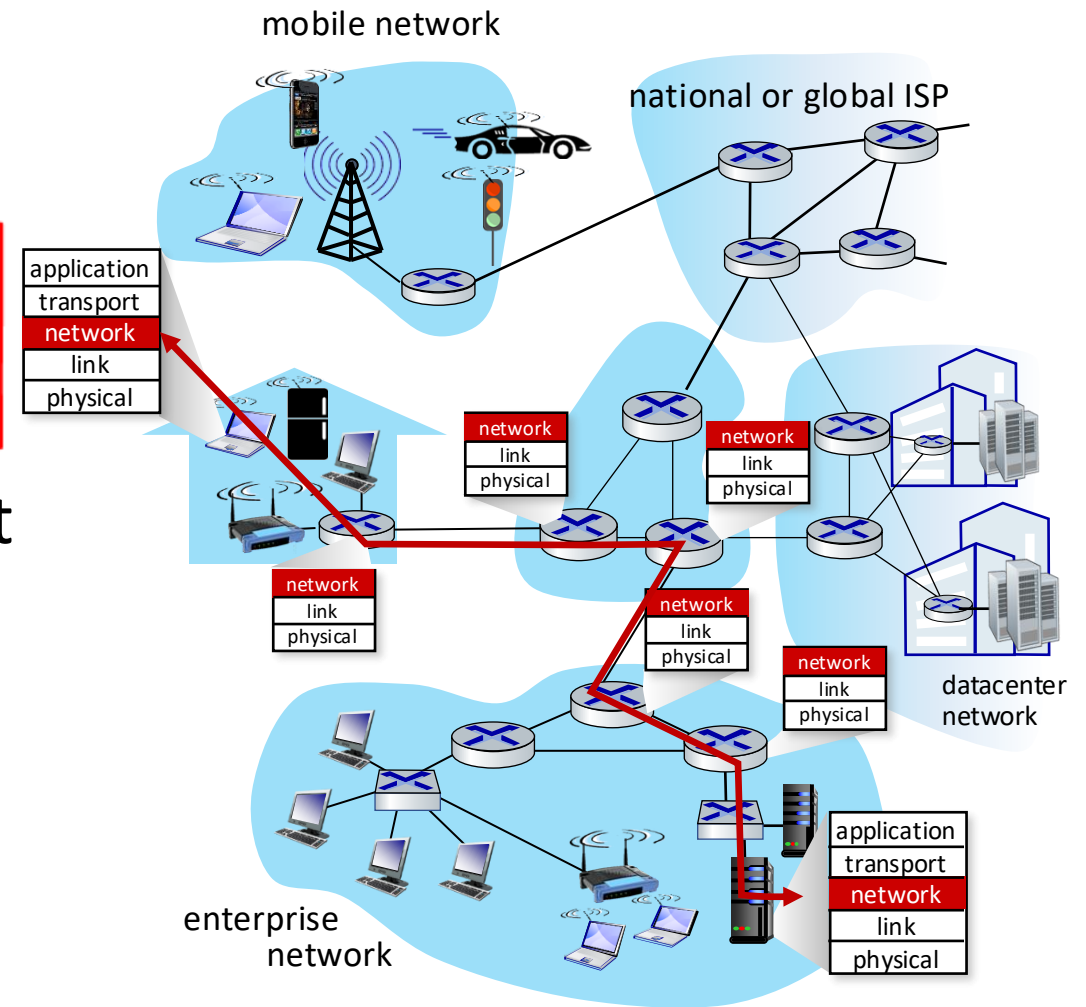
Two approaches to structuring network control plane:

- per-router control (traditional)
- logically centralized control (software defined networking)

Routing protocols

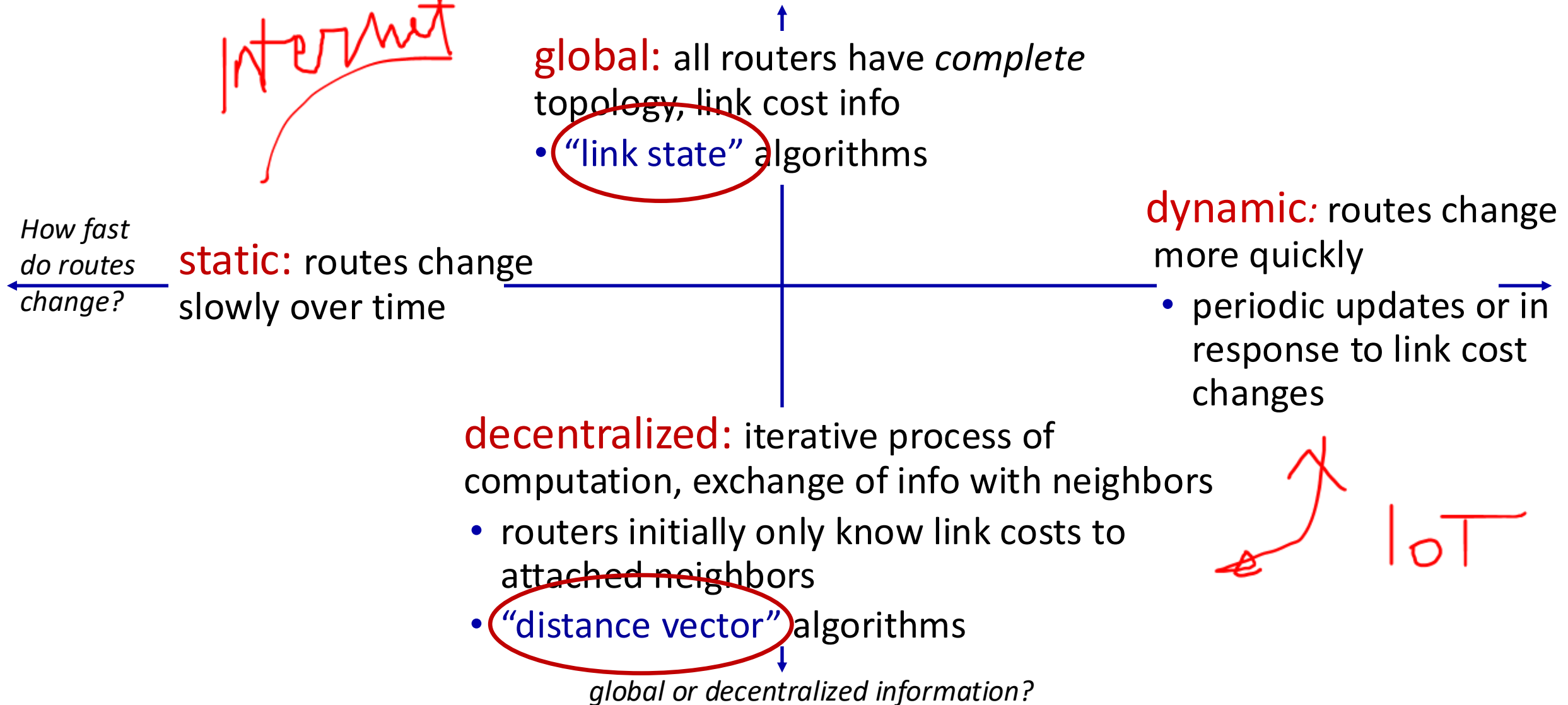
Routing protocol goal: determine “good” paths (equivalently, routes), from sending hosts to receiving host, through network of routers

- **path:** sequence of routers packets traverse from given initial source host to final destination host
- **“good”:** least “cost”, “fastest”, “least congested”
- routing: a “top-10” networking challenge!



Routing algorithm classification

Internet



Recall DSR vs AODV Routing

Aspect	DSR	AODV
Routing method	Source routing, full path carried in packet header	Hop-by-hop routing using routing tables
Packet overhead	Higher, header grows with route length	Lower, route not included in packet header
Route storage	Uses route cache, can keep multiple routes	Uses routing table, usually one next hop per destination
Route freshness	Cached routes can become stale	Uses destination sequence numbers for fresher routes
Main tradeoff	More flexible, but larger headers and stale-cache issues	More efficient forwarding, but unused routes may expire

Wired vs Wireless Routing

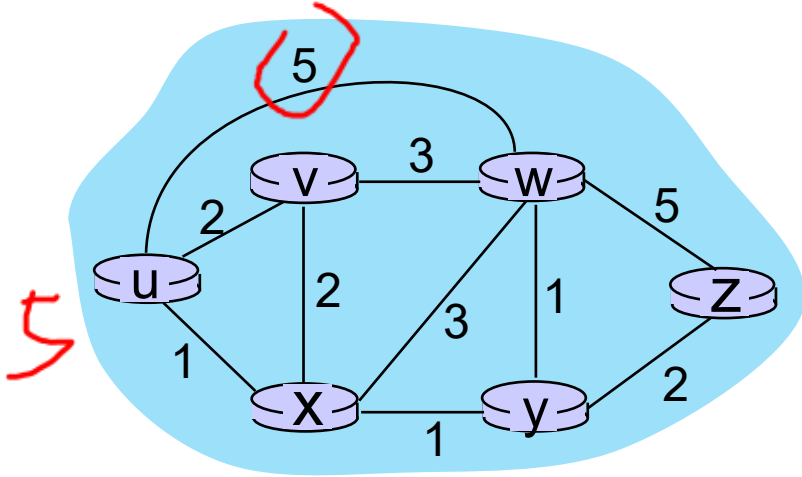
Feature	Wired Routing Protocols	Wireless Routing Protocols
Routing Algorithm	Uses link-state (Dijkstra) or distance vector (Bellman-Ford)	Uses on-demand (AODV, DSR) or proactive (OLSR)
Topology	Static and rarely changes	Dynamic, frequently changes due to mobility
Bandwidth	High bandwidth and dedicated links	Limited bandwidth, affected by interference
Latency	Lower latency due to stable connections	Higher and variable latency due to signal losses
Scalability	Scales well in large networks (BGP, OSPF)	Scalability is a challenge due to routing overhead
Security	More secure (wired access required)	Less secure (wireless signals can be intercepted)

Network layer: “control plane” roadmap

- routing protocols
 - link state - Dijkstra ✓
 - distance vector – Bellman Ford
- Routing algorithms
 - OFPF
 - BGP
- IoT Routing Protocols
 - 6LOWPAN
 - RPL



Graph abstraction: link costs



$c_{a,b}$: cost of *direct* link connecting a and b

e.g., $c_{w,z} = 5$, $c_{u,z} = \infty$

cost defined by network operator:
could always be 1, or inversely related
to bandwidth, or inversely related to
congestion

graph: $G = (N, E)$

N : set of routers = $\{ u, v, w, x, y, z \}$

E : set of links = $\{ (u,v), (u,x), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z) \}$

Dijkstra's link-state routing algorithm

- **global info** : network topology, link costs known to *all* nodes
 - accomplished via “link state broadcast”
 - all nodes have same info
- computes least cost paths from one node (“source”) to all other nodes
 - gives *forwarding table* for that node
- **iterative**: after k iterations, know least cost path to k destinations

notation

- $c_{x,y}$: direct link cost from node x to y ; $= \infty$ if not direct neighbors
- $D(v)$: *current* estimate of cost of least-cost-path from source to destination v
- $p(v)$: predecessor node along path from source to v
- N' : set of nodes whose least-cost-path *definitively* known

Dijkstra's link-state routing algorithm

1 *Initialization:*

2 $N' = \{u\}$ /* compute least cost path from u to all other nodes */

3 for all nodes v

4 if v adjacent to u /* u initially knows direct-path-cost only to direct neighbors */

5 then $D(v) = c_{u,v}$ /* but may not be *minimum* cost! */

6 else $D(v) = \infty$

7



8 *Loop*

9 find w not in N' such that $D(w)$ is a minimum

10 add w to N'

11 update $D(v)$ for all v adjacent to w and not in N' :

12 **$D(v) = \min (D(v), D(w) + c_{w,v})$**

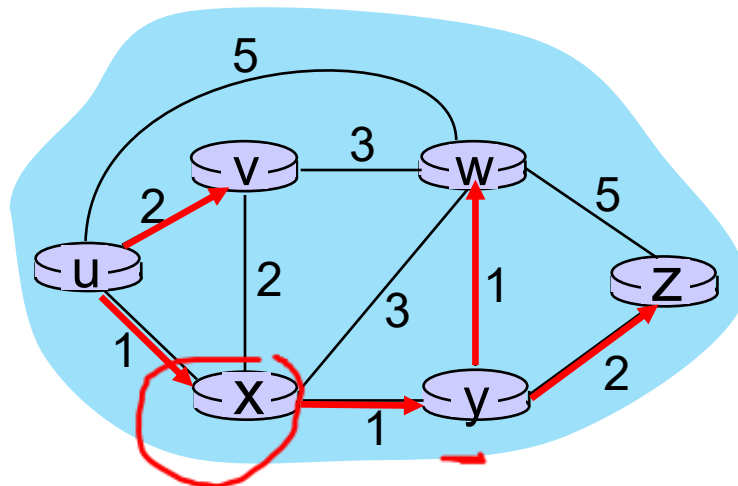
13 /* new least-path-cost to v is either old least-cost-path to v or known

14 least-cost-path to w plus direct-cost from w to v */

15 *until all nodes in N'*

Dijkstra's algorithm: an example

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	uxyv		3, y			4, y
4	uxyvw					4, y
5	uxyvwz					



Initialization (step 0): For all a : if a adjacent to u , then $D(a) = c_{u,a}$

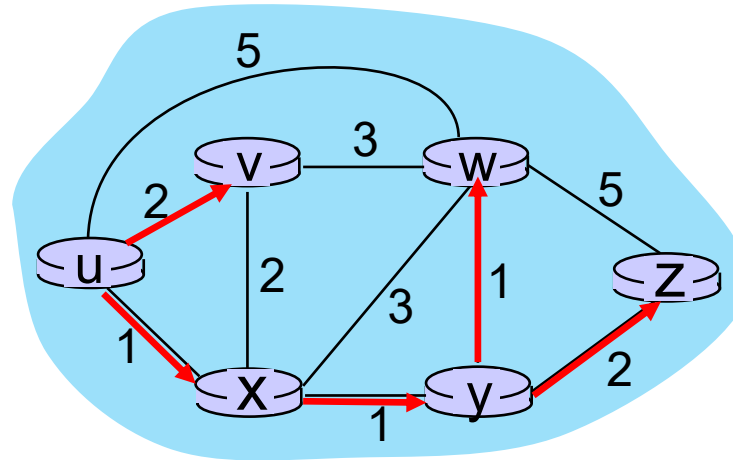
find a not in N' such that $D(a)$ is a minimum

add a to N'

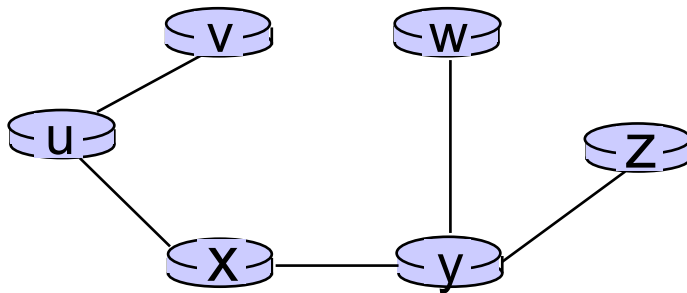
update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

Dijkstra's algorithm: an example



resulting least-cost-path tree from u:



resulting forwarding table in u:

destination	outgoing link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

route from u to v directly

route from u to all other destinations via x

Dijkstra's algorithm: discussion

algorithm complexity: n nodes

- each of n iterations: need to check all nodes, w , not in N
- $n(n+1)/2$ comparisons: $O(n^2)$ complexity
- more efficient implementations possible: $O(n \log n)$

message complexity:

- each router must *broadcast* its link state information to other n routers
- efficient (and interesting!) broadcast algorithms: $O(n)$ link crossings to disseminate a broadcast message from one source
- each router's message crosses $O(n)$ links: overall message complexity: $O(n^2)$

Internet approach to scalable routing

aggregate routers into regions known as “autonomous systems” (AS) (a.k.a. “domains”)

intra-AS (aka “intra-domain”):
routing among *within same AS*
(“network”)

- all routers in AS must run same intra-domain protocol
- routers in different AS can run different intra-domain routing protocols
- **gateway router**: at “edge” of its own AS, has link(s) to router(s) in other AS'es

inter-AS (aka “inter-domain”):
routing *among* AS'es

- gateways perform inter-domain routing (as well as intra-domain routing)

Network layer: “control plane” roadmap

- routing protocols
 - link state - Dijkstra
 - distance vector – Bellman Ford
- Routing algorithms
 - OFPF
 - BGP
- IoT Routing Protocols
 - 6LOWPAN
 - RPL

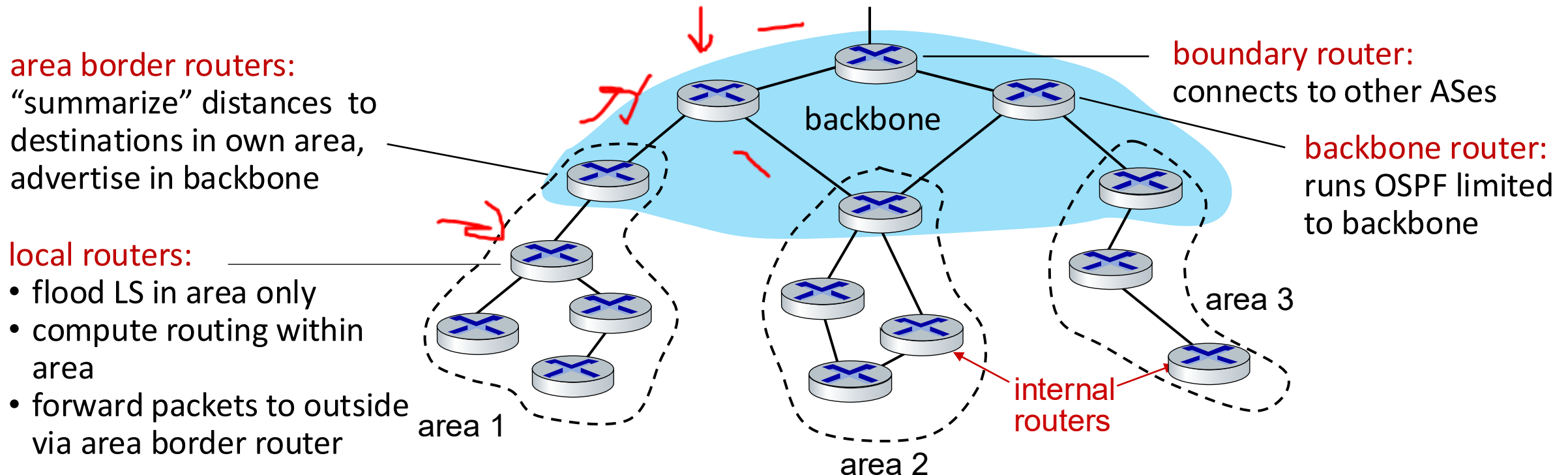


OSPF (Open Shortest Path First) routing

- “open”: publicly available
- classic link-state
 - each router floods OSPF link-state advertisements (directly over IP rather than using TCP/UDP) to all other routers in entire AS
 - multiple link costs metrics possible: bandwidth, delay
 - each router has full topology, uses Dijkstra’s algorithm to compute forwarding table
- *security*: all OSPF messages authenticated (to prevent malicious intrusion)

Hierarchical OSPF

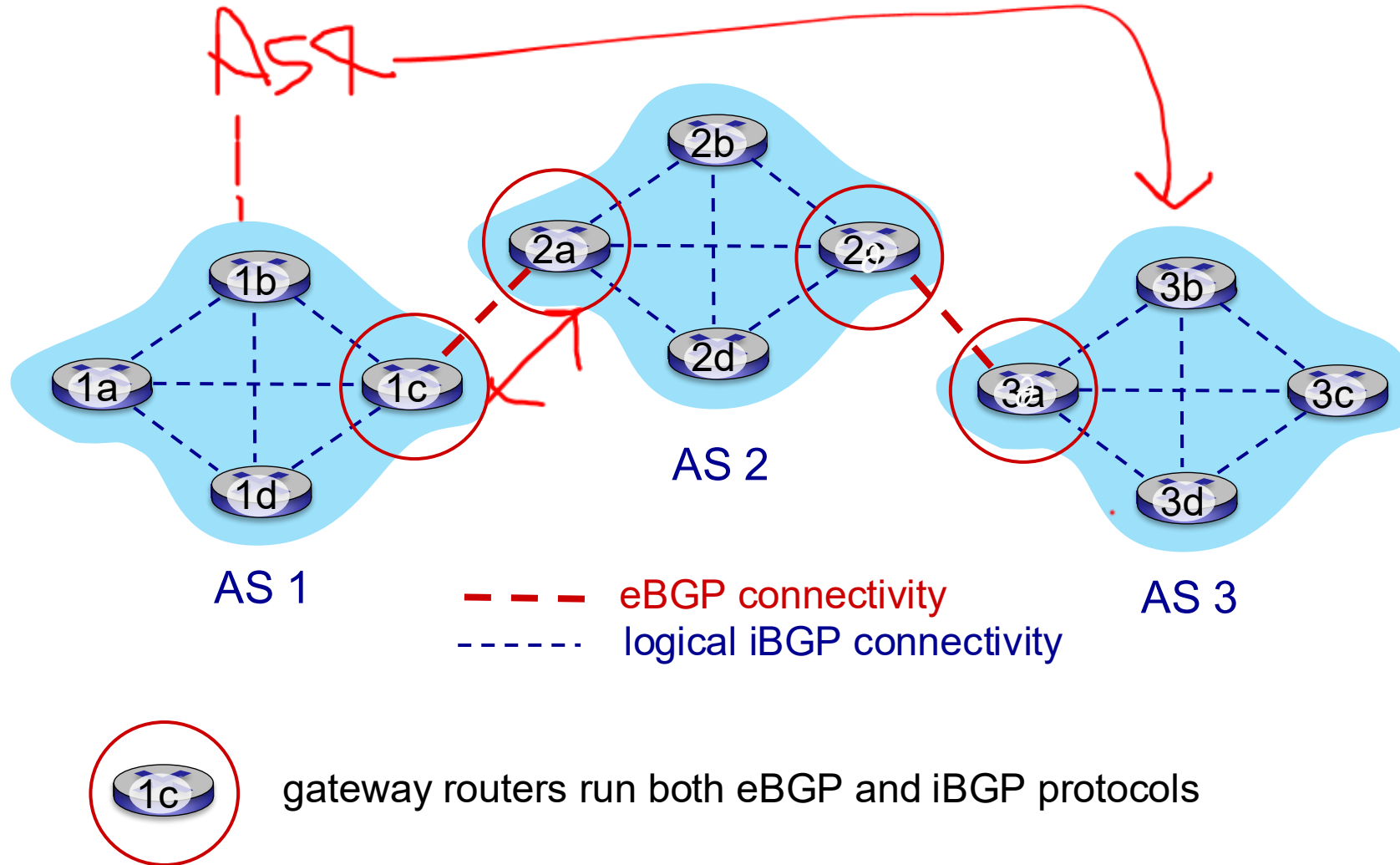
- **two-level hierarchy:** local area, backbone.
 - link-state advertisements flooded only in area, or backbone
 - each node has detailed area topology; only knows direction to reach other destinations



Internet inter-AS routing: BGP

- **BGP (Border Gateway Protocol):** *the* de facto inter-domain routing protocol
 - “glue that holds the Internet together”
 - allows subnet to advertise its existence, and the destinations it can reach, to rest of Internet: *“I am here, here is who I can reach, and how”*
 - BGP provides each AS a means to:
 - **eBGP:** obtain subnet reachability information from neighboring ASes
 - **iBGP:** propagate reachability information to all AS-internal routers.
 - determine “good” routes to other networks based on reachability information and *policy*
-

eBGP, iBGP connections



Why different Intra-, Inter-AS routing ?

policy:

- inter-AS: admin wants control over how its traffic routed, who routes through its network
- intra-AS: single admin, so policy less of an issue

scale:

- hierarchical routing saves table size, reduced update traffic

performance:

- intra-AS: can focus on performance
- inter-AS: policy dominates over performance

Network layer: “control plane” roadmap

- routing protocols
 - link state - Dijkstra
 - distance vector – Bellman Ford
- Routing algorithms
 - OFPF
 - BGP
- IoT Routing Protocols
 - 6LOWPAN
 - RPL



6LowPAN

1. 6LowPAN

- **References**

- Olivier Hersent, David Boswarthick and Omar Elloumi, The Internet of Things: Key Applications and Protocols, 2nd ed., Wiley, 2012. **(E-BOOK)**
 - Chapter 12
- Additional References posted on LMS under “Reading Material”

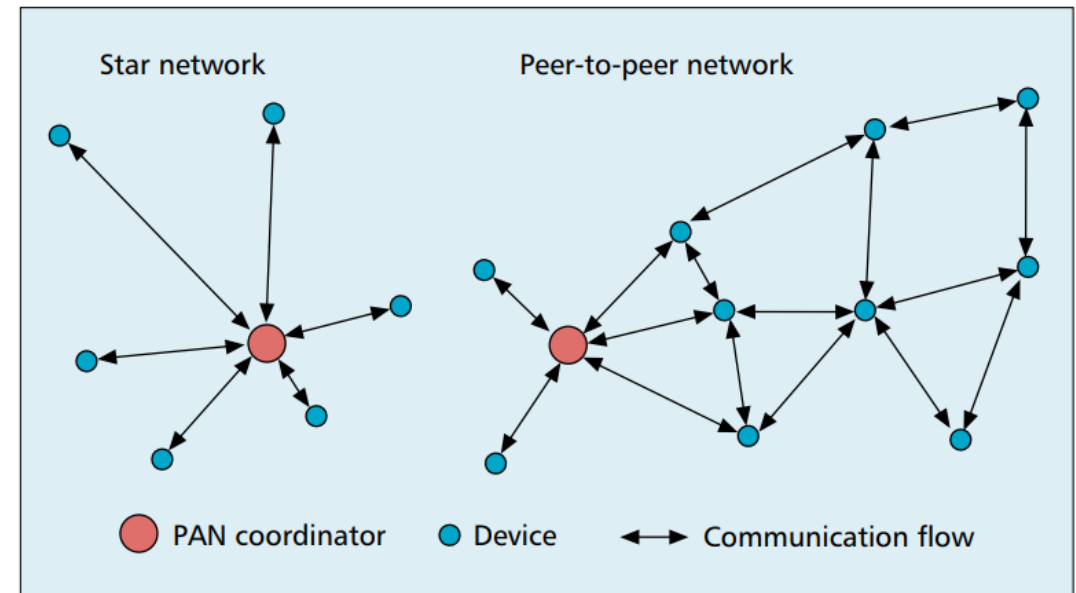
Network and Routing Layer/Protocols

Application Protocol		DDS	CoAP	AMQP	MQTT	MQTT-SN	XMPP	HTTP REST
Service Discovery		mDNS				DNS-SD		
Infrastructure Protocols	Routing Protocol	RPL						
	Network Layer	6LoWPAN				IPv4/IPv6		
	Link Layer	IEEE 802.15.4						
	Physical/ Device Layer	LTE-A	EPCglobal		IEEE 802.15.4		Z-Wave	
Influential Protocols		IEEE 1888.3, IPsec				IEEE 1905.1		

[Fig. Source: A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari and M. Ayyash, "Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications," in IEEE Communications Surveys & Tutorials, vol. 17, no. 4.]

Recap: IEEE 802.15.4 Core Features

- Wireless Personal Area Network (WPAN) standard
- Allows both piconet and scatternet (mesh networking) topologies
- Full function nodes (routers) can forward packets to other nodes
- One full function node acting at the PAN coordinator
- Nodes have 64-bit link layer addresses
- PAN coordinator assigns 16-bit short address to nodes (only valid inside the PAN)
- Does not support IP addresses or routing
 - Motivation for 6LowPAN



Addresses

Ethernet addresses: 48 bit MAC

Unicast Multicast	Universal Local	Organizationally Unique ID (OUI)	Manufacturer Assigned
1b	1b	22b	24b

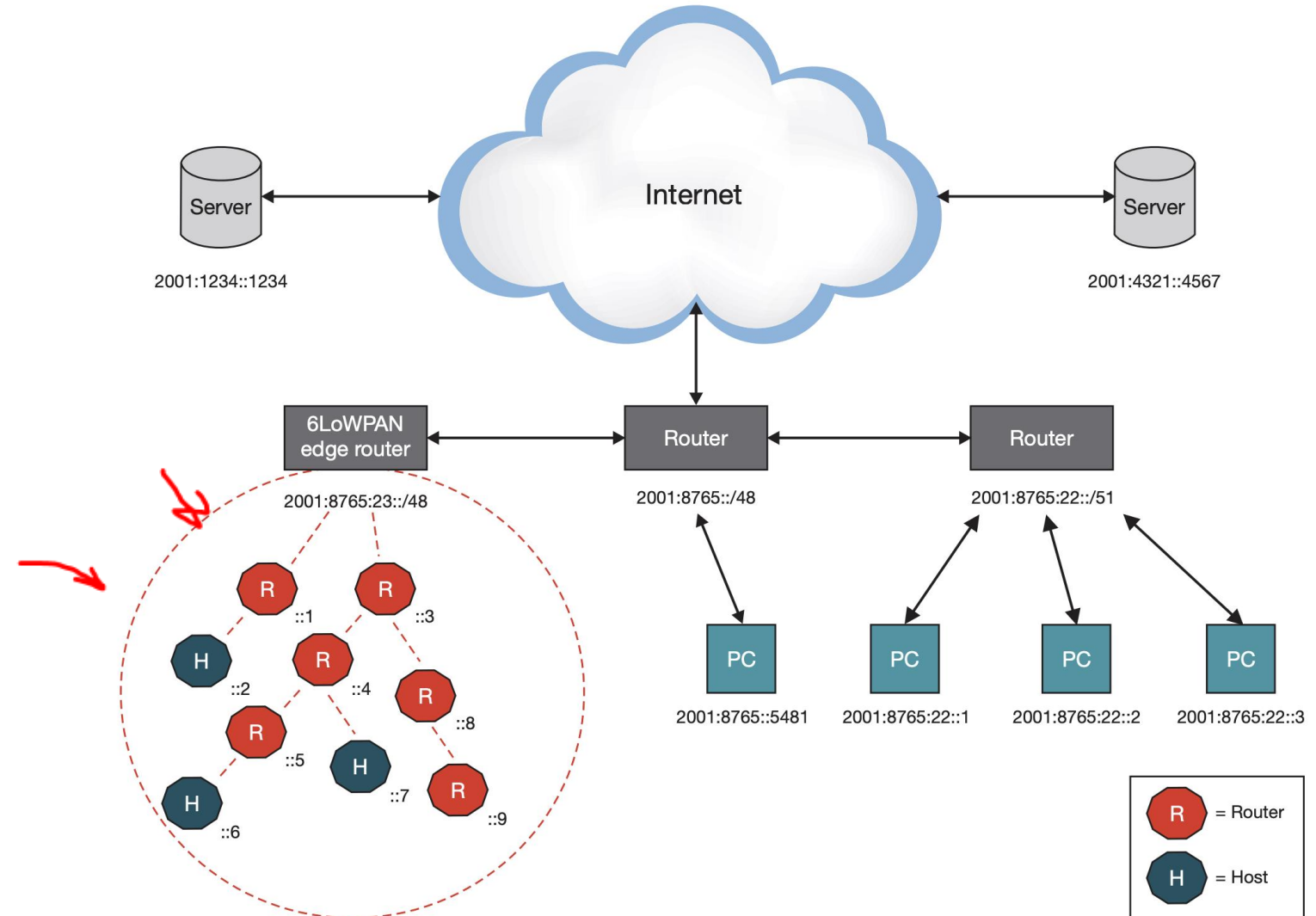
IEEE 802.15.4 Addresses: 64 bit Extended Unique Id (EUI)

Unicast Multicast	Universal Local	Organizationally Unique ID (OUI)	Manufacturer Assigned
1b	1b	22b	40b

Include 16-bit PAN ID

- IEEE 802.15.4 allows 16-bit short addresses (locally significant), instead of 64-bit EUI
- The PAN ID is a 16-bit identifier for the whole 802.15.4 network.
- IPv6 Link Local Address can be formed as
 - FE80 :: PAN ID : Short Address
 - Use 0 for PAN ID if unknown

6LoWPAN basics

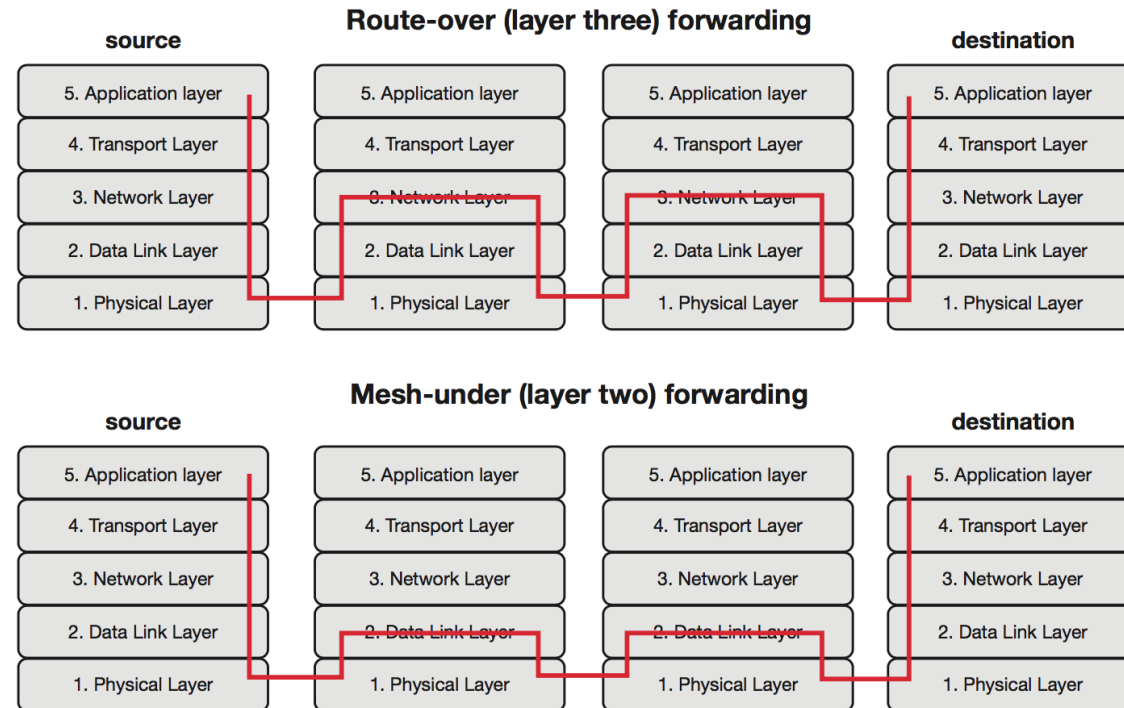


References

- IETF RFC4919: "IPv6 over Low-Power Wireless Personal Area Networks (6LoWPANs): Overview, Assumptions, Problem Statement, and Goals".
- IETF RFC4944: "Transmission of IPv6 Packets over IEEE 802.15.4 Networks".

6LowPAN – two types of routing

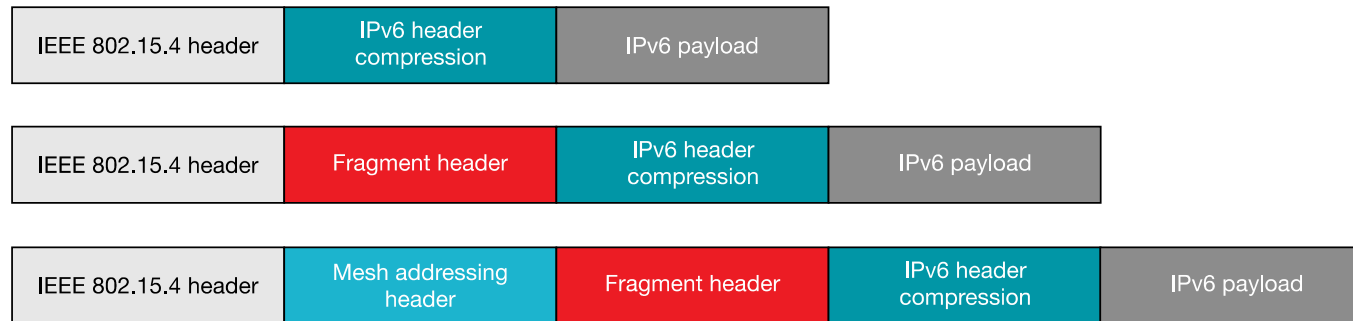
- Supports types of addressing/routing
 - "Mesh under"
 - Layer 2 takes care of routing
 - IP layer sees a large subnet
 - Works only with a single link layer technology
 - "Route over"
 - Layer 3 mesh routing
 - Multiple link layer technologies can be used



6LowPAN core features

- Two key issues
 - Header compression
 - IPv6 header takes up 40 bytes even without extension header
 - Fragmentation and reassembly
 - IEEE 802.15.4 only allows up to 127 bytes of payload
- 

6LoWPAN Headers



[Fig. Source: J. Olsson, "6LoWPAN Demystified", Texas Instruments]

RPL (“Ripple”)

Reference

Vasseur, J and Agarwal, Navneet and Hui, Jonathan and Shelby, Zach and Bertrand, Paul and Chauvenet, Cedric, “RPL: The IP routing protocol designed for low power and lossy networks,” Internet Protocol for Smart Objects (IPSO) Alliance, v. 36, 2011.

❖ Additional Reading:

- Olivier Hersent, David Boswarthick and Omar Elloumi, The Internet of Things: Key Applications and Protocols, 2nd ed., Wiley, 2012. **(E-BOOK)**
 - Chapter 12

Low power Lossy Networks (LLNs)

- Resource constrained nodes and links
 - processing
 - memory
 - battery
- Both links and routers are constrained
- Scale from dozen to thousands of nodes
- Unreliable links
 - variable loss rates
 - low data rates
 - unstable links

Routing for LLNs (ROLL)

- ROLL: IETF Working Group to standardize an IPv6-based routing for LLNs
- Conducted a detailed analysis of routing protocols for:
 - urban networks e.g. smart grid
 - industrial automation
 - home and building automation
- Low bandwidth/power, Lossy Networks
 - The new protocol should work with any LL: 802.15.4, WiFi, PowerLine Communication (PLC), etc.
- The result was RPL

Other routing protocols

- Protocols evaluated under five criteria
 - Routing State – does it ~~work with limited~~ memory resources of low-power nodes?
 - Loss Response – does it handle link failures well?
 - Control cost – does the control cost overhead scale?
 - Link & Node cost – can it handle link and node properties (eg desirable metrics) when choosing routes?

Protocol	State	Loss	Control	Link Cost	Node Cost
OSPF/IS-IS	fail	fail	fail	pass	fail
OLSRv2	fail	?	?	pass	pass
TBRPF	fail	pass	fail	pass	?
RIP	pass	fail	pass	?	fail
AODV	pass	fail	pass	fail	fail
DYMO	pass	?	pass	?	?
DSR	fail	pass	pass	fail	fail

[Fig. Source: S. Kuryla, "RPL: IPv6 Routing Protocol for Low Power and Lossy Networks,"]

RPL

- A distance vector IPv6 routing protocol for LLN
- IETF standard in 2012 <https://tools.ietf.org/html/rfc6550>
- Builds a **logical Destination Oriented Directed Acyclic Graph (DODAG)** using
 - an **objective function**
 - a set of **metrics/constraints**
- To compute the best path
 - The logical graph may (and usually is) different from the physical connectivity graph
 - Can support multiple graphs at the same time on the same network (to carry traffic according to different objectives)

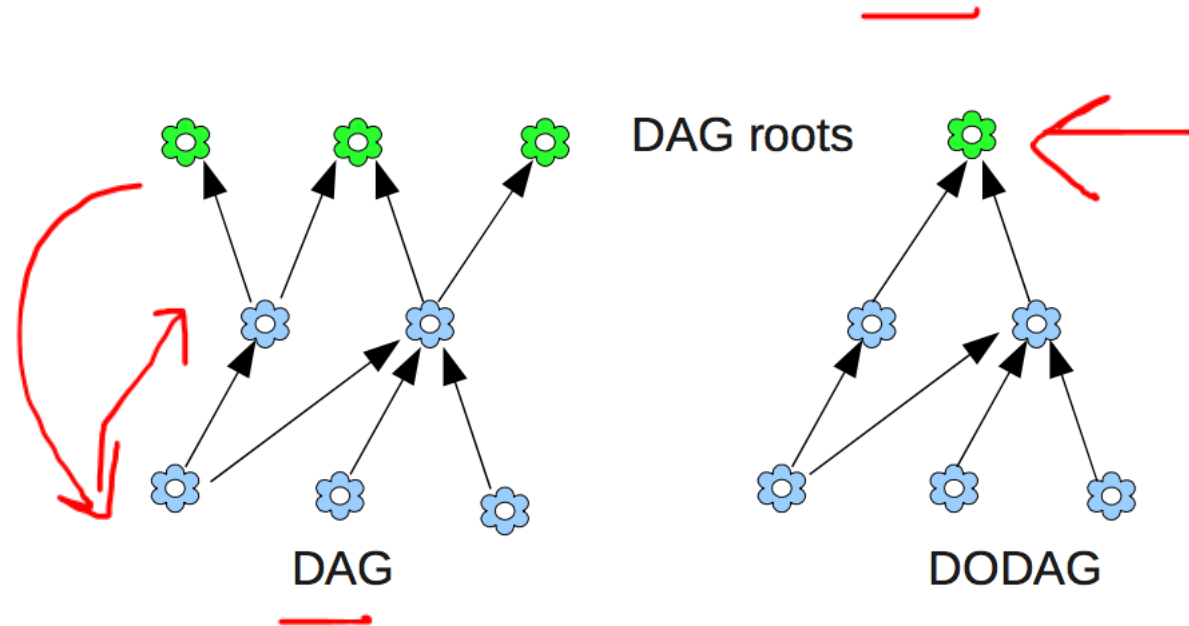
Examples of RPL graph objectives

- (1) 'Find paths with best ETX [Expected Transmissions] values (metric) and avoid non-encrypted links (constraint)' or
- (2) 'Find the best path in terms of latency (metric) while avoiding battery-operated nodes (constraint)'
- A node in the network can participate and join one or more
 - graphs, in this case called “RPL instances”
 - marked traffic flows up and down along the edges of the specific graph (instance)

RPL: DAG and DODAG

- DAG and DODAG

- DAG: Directed Acyclic Graph
- DODAG: Destination Oriented Directed Acyclic Graph



DAG/DODAG definitions

- **DAG:** Directed Acyclic Graph.

- A directed graph having the property that all edges are oriented in such a way that no cycles exist. All edges are contained in paths oriented toward and terminating at one or more root nodes.

- **DAG root:**

- A DAG root is a node within the DAG that has no outgoing edge. Because the graph is acyclic, by definition, all DAGs must have at least one DAG root and all paths terminate at a DAG root.

- **Destination-Oriented DAG (DODAG):**

- A DAG rooted at a single destination, i.e., at a single DAG root (the DODAG root) with no outgoing edges.

- **DODAG root:**

- A DODAG root is the DAG root of a DODAG. The DODAG root may act as a border router for the DODAG; in particular, it may aggregate routes in the DODAG and may redistribute DODAG routes into other routing protocols.

- **DODAGID:**

- A DODAGID is the identifier of a DODAG root. The DODAGID is unique within the scope of a RPL Instance in the LLN. The tuple (RPLInstanceID, DODAGID) uniquely identifies a DODAG.

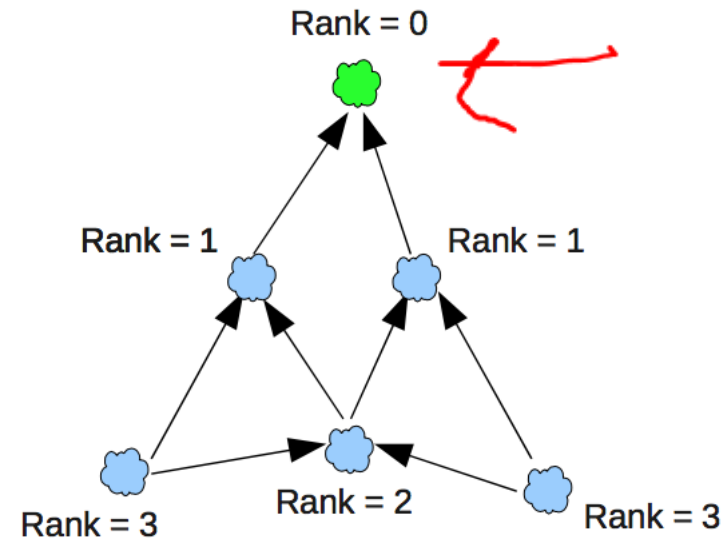
- **An OF (Objective Function):**

- defines how routing metrics, optimization objectives, and related functions are used to compute Rank. Furthermore, the OF dictates how parents in the DODAG are selected and, thus, the DODAG formation.

RPL: Node Rank

■ Node Rank

- defines the node's individual position relative to other nodes with respect to a DODAG root
- is computed based on the DAG's OF.
- could be computed simply based on topological distance, or based on OF's link/node metrics and constraints



Routing Attributes

- Node State and Attributes
 - Node workload or resources (CPU, Memory, ...)
- Node Energy -
- Hop Count -
- Throughput -
- Latency
- Link Reliability
- Other admin defined metrics

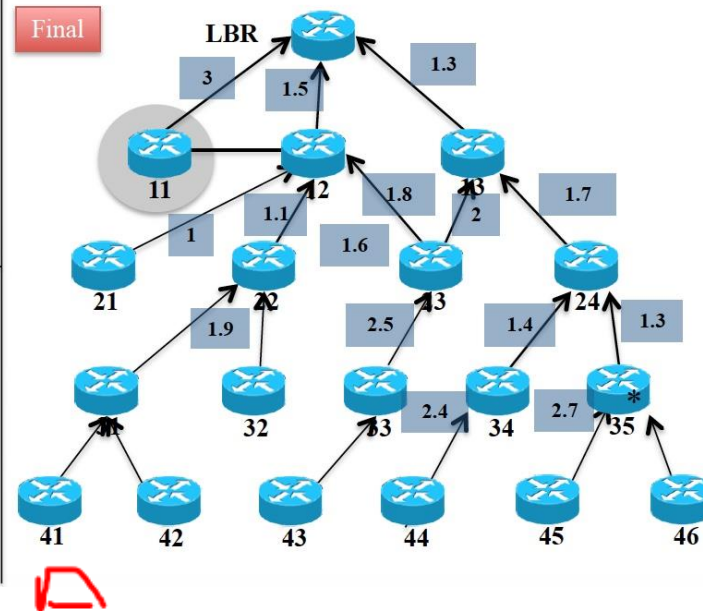
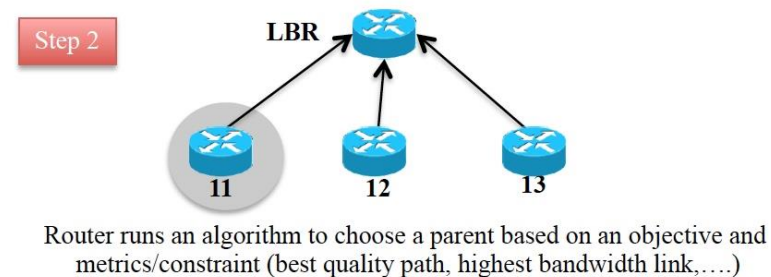
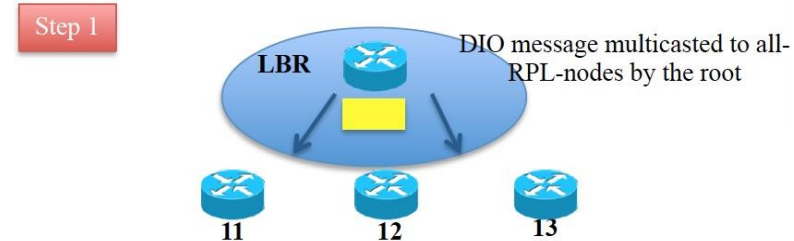
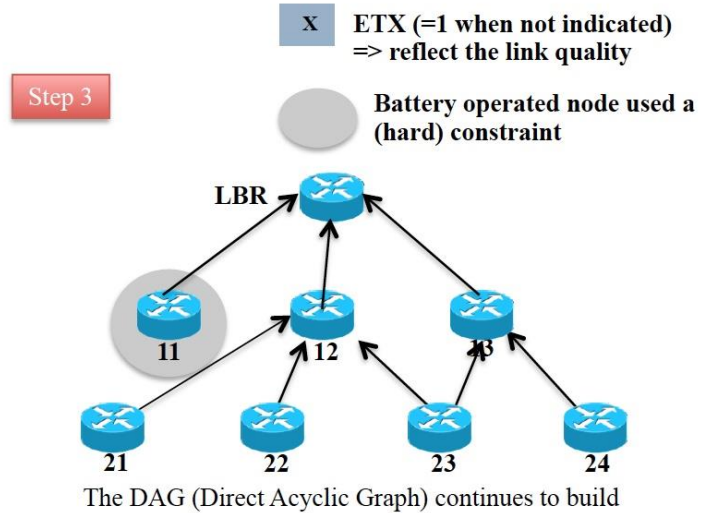
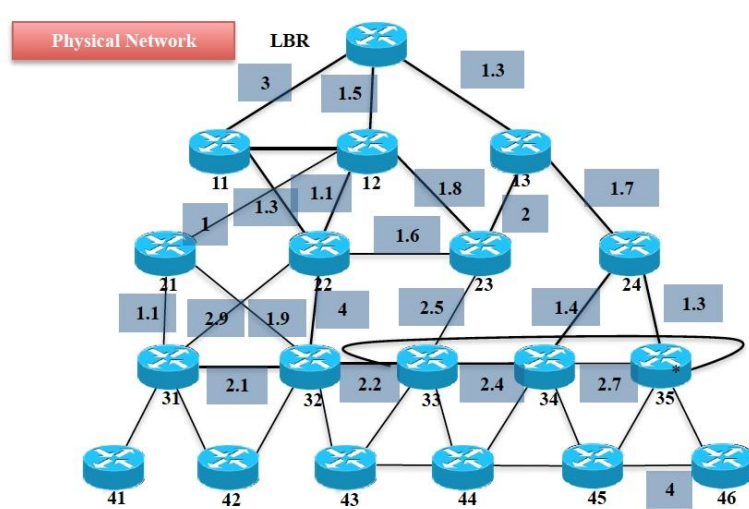
RPL Control Messages

- RPL defines new ICMPv6 messages (three types):
 - DAG **Information Object** (DIO)
 - Announces an RPL Instance along with its configuration parameters and helps nodes select DODAG parent
 - **Propagated downwards** in DODAG in support of upward traffic to the root
 - DAG **Information Solicitation** (DIS)
 - Solicit a DODAG Information Object (ie a RPL instance) from a RPL (neighbor) node
 - Destination **Advertisement Object** (DAO)
 - **Propagated upwards** along the DODAG about the reachability of leaf nodes, in support of the downward traffic flow.

DODAG Construction

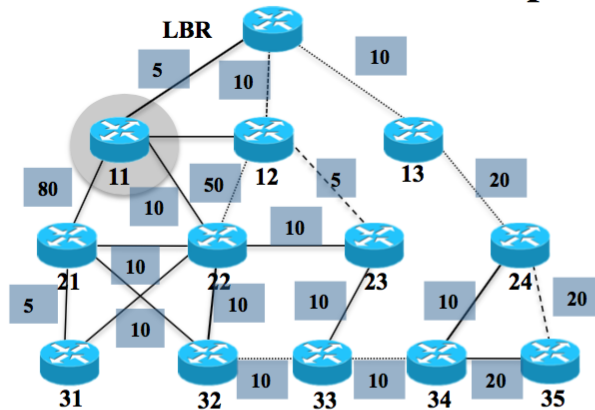
- Nodes periodically send link-local multicast DIO messages
- Nodes listen for DIOs and use their information to join a new DODAG, or to maintain an existing DODAG
- Nodes may use a DIS message to solicit a DIO
 - e.g. if they join an already formed DODAG
- Based on information in the DIOs the node chooses parents that minimize path cost to the DODAG root
- Result: an MP2P (**Multipoint-to-point**) forwarding model where each node of the graph has reach-ability toward the graph root.

DODAG Construction Example

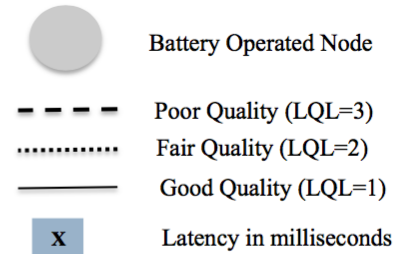


RPL Instances

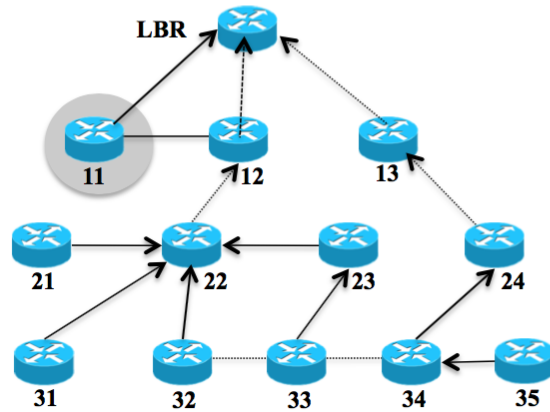
Physical topology



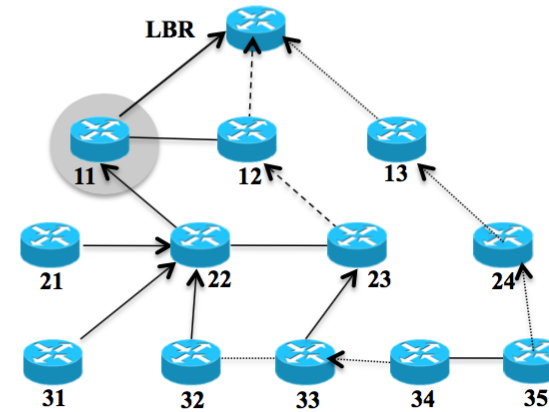
Concept of Multiple RPL Instances (a la MTR)



DAG Instance 1: high quality – no battery operated nodes
DAG Instance 2: low latency



RPL instance 1



RPL instance 2

RPL: Traffic Flows

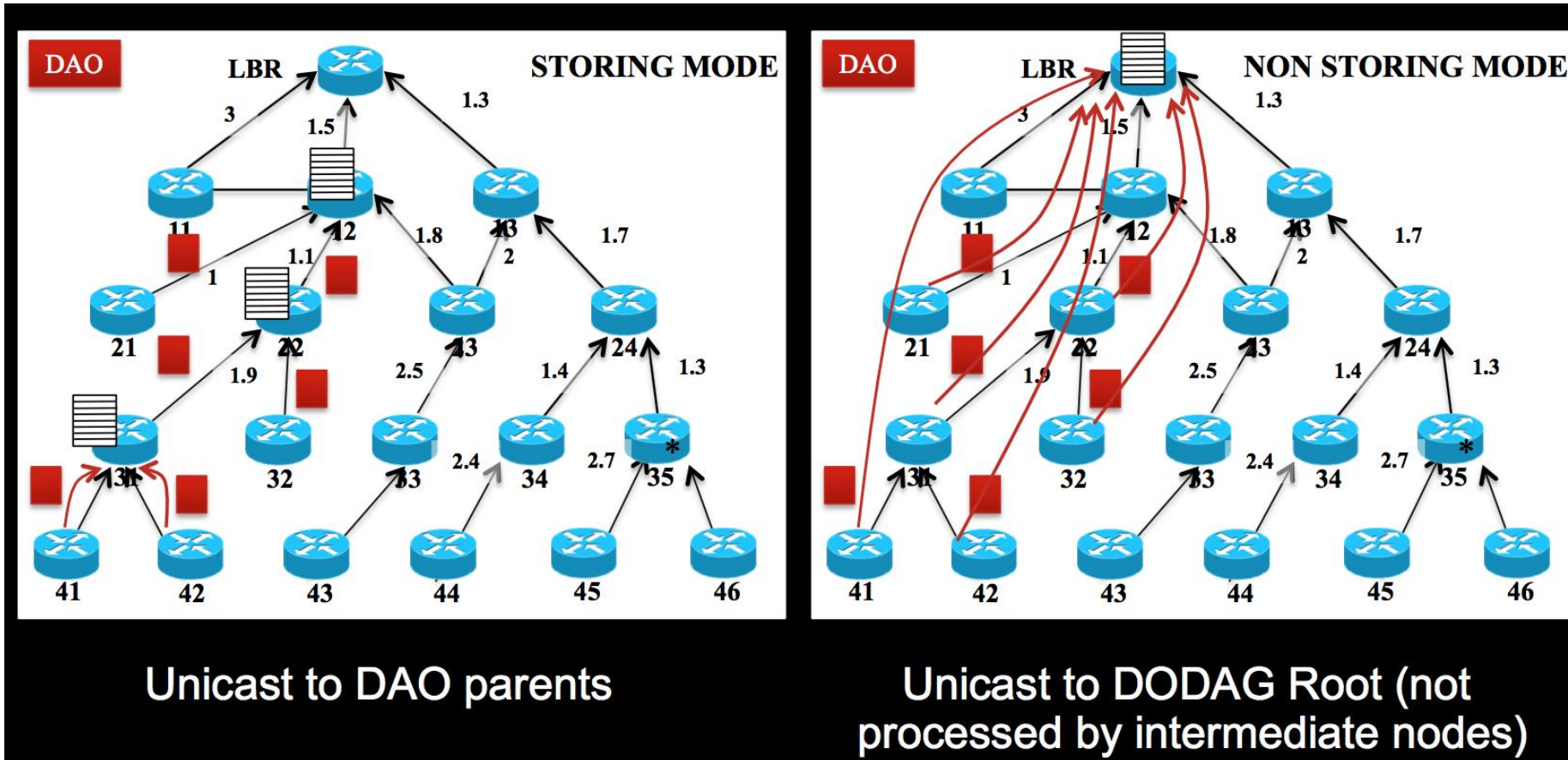
- One-to-many
 - Root to all other nodes (downward)
- Many-to-one
 - All other nodes to root (upward)
- One-to-one
 - First upward, then downward
 - Goes through root, in the worst case (root knows about all nodes on the DODAG)

Additional Slides: RPL Link Failure

RPL: Storing vs Non-storing modes

- A storing LLN
 - Each intermediate node has available memory to store routing tables
 - Utilizes the Dest. Adv. Obj. (DAO) messages
 - Keeps a downward routing table at each node
 - Supports downward traffic towards the leaves
 - Traffic travels only as far as common parent
 - Limited by size of routing table
 - protocol fails when any table is full
- A non-storing LLN sends all traffic to root (upward)
 - Root uses source routes to send traffic to leaves (downward)
- A mixed mode of operation is not allowed and all nodes in the graph have to operate in either storing or non-storing mode only

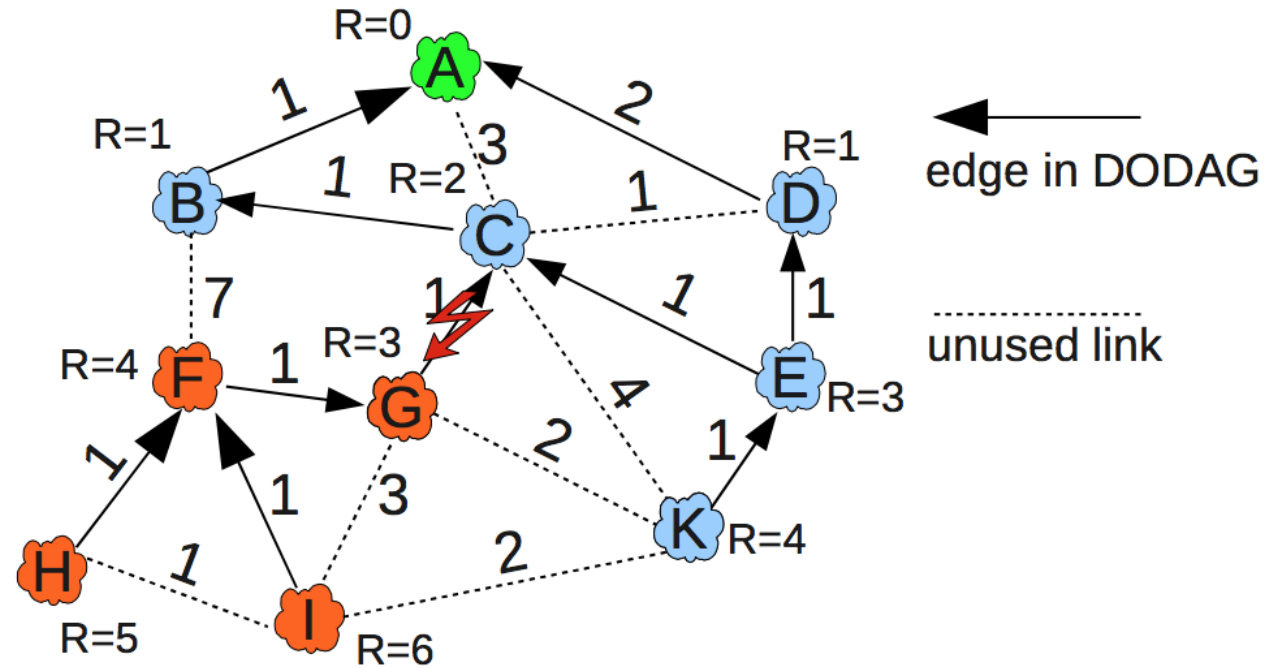
RPL: Storing vs Non-storing Modes



[Fig. Source: JP Vasseur, "IoT Workshop RPL Tutorial," April 2011, © Cisco Systems]

RPL Link Failure

- Link between nodes G and C fails
- Two repair mechanisms
 - Global (involves the root) vs Local



RPL Global vs Local Repair

- Global Repair
 - New DODAG version
 - Triggered by the root
- Local Repair
 - Local and quick
 - Failed node does (Dag Info Obj) DIO-poisoning (advertise INFINITE_RANK)
 - Allowed to choose a new parent who is within a pre-defined threshold (MaxRankIncrease) of the old Rank

RPL Local Repair Example

